

COURSE NAME

SOFTWARE QUALITY
AND TESTING

CSC 4133

(UNDERGRADUATE)

CHAPTER 4

SOFTWARE QUALITY ENGINEERING

Dr. Mohammad Mahmudul Hasan

Associate Professor

Department Of Computer Science

American International University – Bangladesh

Email: m.hasan@aiub.edu

Web: <http://cs.aiub.edu/profile/m.hasan>



Google Scholar



LinkedIn

ORCID

WEB OF SCIENCE™



QUALITY ASSURANCE

- ❑ QA focus on **correctness** aspect of quality, and dealing with **defects**
- ❑ QA activities can be classified into three generic categories –
 - **Defect Prevention:** QA activities **prevent** certain types of faults from being injected into the software since errors are the missing/incorrect human actions that lead to injection of faults into software systems (**default values, options to select**)
 - **Defect Reduction:** QA activities detect and **remove** certain faults once they have been injected into the software systems.
 - **Defect Containment:** control defect through **fault tolerance**, failure prevention, or failure **impact minimization** to assure software reliability and safety (e.g., **auto save option**)

DEFECT PREVENTION

Defect prevention can be done by two generic ways:

❑ Error source removal (known error sources)

- Eliminating certain error sources such as eliminating ambiguities or correcting human misconceptions, which are the root causes for errors and remove through education and training (people-based solution)

❑ Error Blocking

- Fault prevention or blocking by directly correcting or blocking these missing or incorrect human actions (e.g., user cannot give 0 value to the divisor attribute; data validation in excel)
- This will result the direct intervention to block errors (fault injections prevented)
- Systematic defect prevention via process improvements
- If imprecise designs and implementations, tools and technology that deviate from product specifications or design intentions are the causes for faults, formal methods can help prevent such deviations

DEFECT REDUCTION

- ❑ It is unrealistic to expect the defect prevention activities to be 100% effective in preventing accidental fault injections. Following are the techniques to remove as many faults as possible
- ❑ **Inspection Method to Reduce Defects**
 - The most commonly used static technique uses critical reading and analysis of software code, designs, product specifications, test plans, etc.
 - Formality and structures of Inspections varies in
 - **Informal** reviews/walkthroughs (self-conducted, independent, pass around, canteen discussion)
 - **Formal** Inspection are typically conducted by multiple human inspectors through some predefined coordination of process.

DEFECT REDUCTION

❑ Testing Method to Reduce Defects

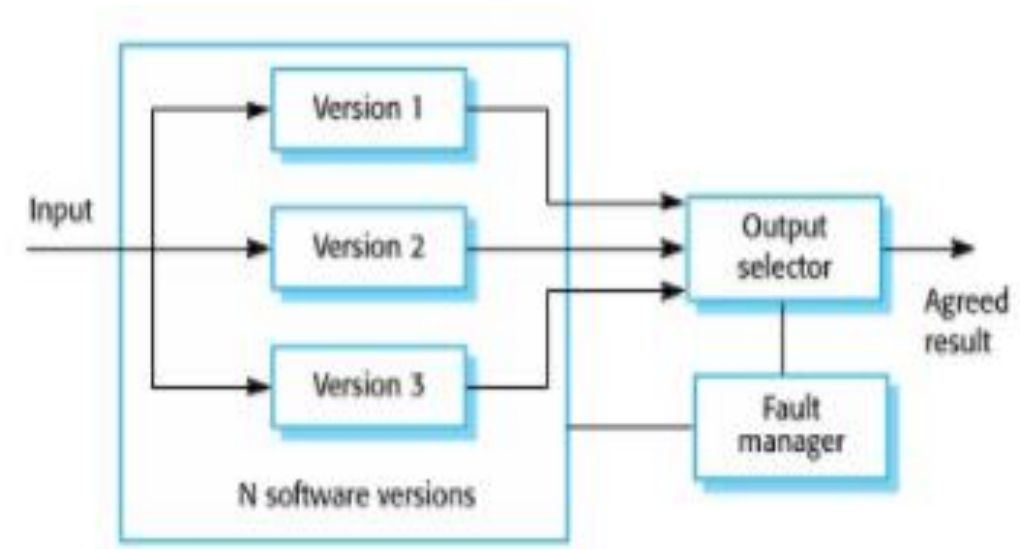
- Involves the execution of software and the observation of the program behavior/outcome
- If a failure is observed, the execution record is then analyzed to locate & fix the fault(s) that caused the failure.

❑ Other techniques and Risk identification Method to Reduce Defects

- Formal model-based analyses
- Boundary value analysis (e.g., loop condition check)
- Control flow and data flow analyses (e.g., conditional statements)
- Simulation and Prototyping (e.g., Autopilot)

DEFECT CONTAINMENT

- ❑ Defect reduction activities can only **reduce the number of faults to a fairly low level**, but **not completely eliminate them** (because of the large size and high complexity of most software systems in use today as **complete testing is not possible**)
- ❑ The containment measures focus on the failures by either containing them to local areas so that there are no global failures observable to users, or limiting the damage caused by software system failures (e.g., **exception handling**)
- ❑ Defect Containment can be done in two generic ways:
 - ❖ **Software fault-tolerance**
 - **Recovery**: rollback and redo
 - **NVP**: N-Version programming (fault blocked)
 - Auto save
 - ❖ **Safety assurance and failure containment**



QA IN SOFTWARE PROCESS

❑ Mega-process

- Initiation
- Development
- Maintenance
- Termination

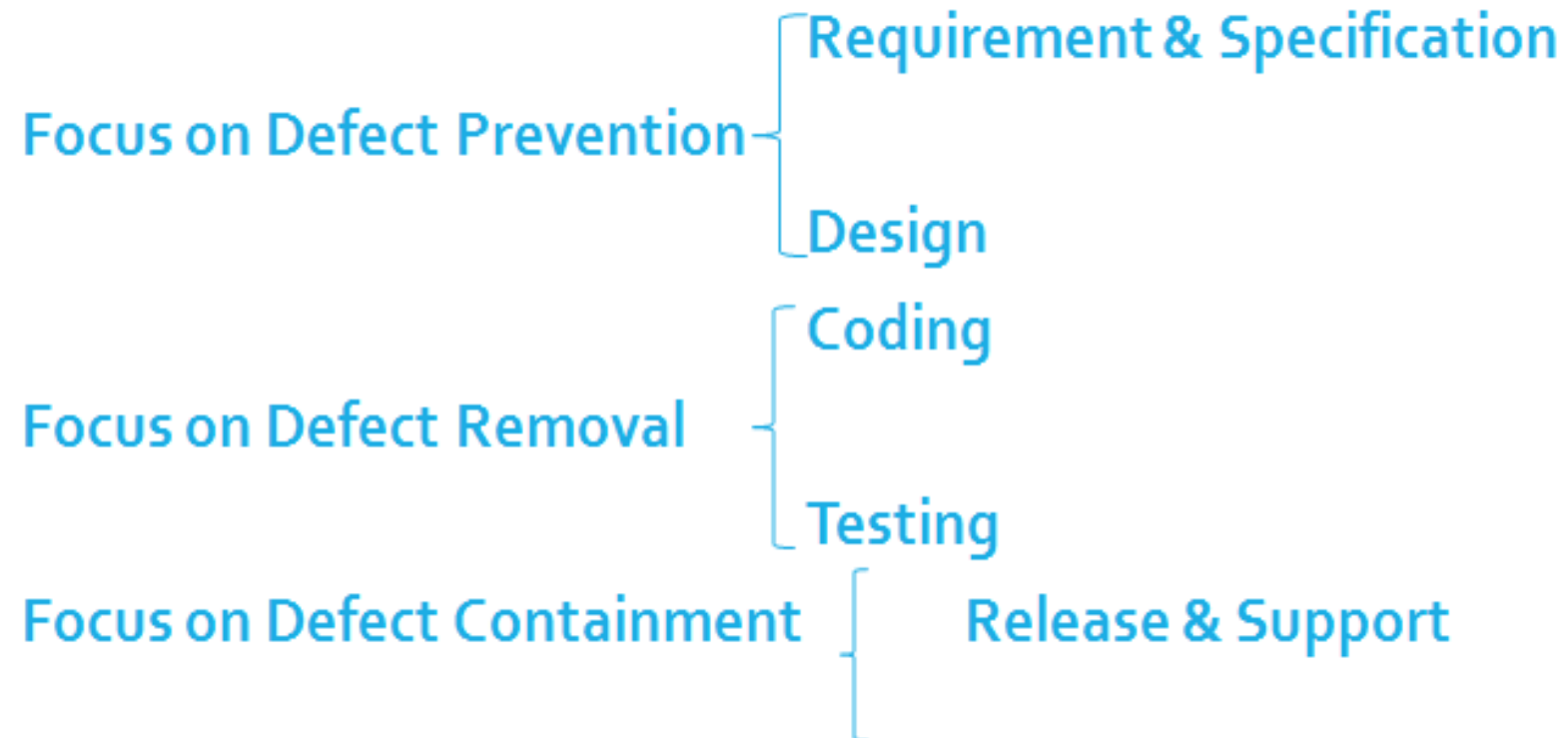
❑ SDLC (Development components)

- Requirement
- Specification
- Design
- Coding
- Testing
- Release
- Maintenance

❑ Process variations

- Waterfall development process
- Iterative development process (QA in increments)
- Spiral development process (QA & risk management)
- Agile development process (XP- test driven development pair programming)
- Maintenance processes (focus on defect handling)

QA IN WATERFALL SOFTWARE PROCESS



QA ACTIVITIES: MAPPING FROM DC VIEW TO V&V VIEW

DC (defect-centered) view	QA activity	Validation(external focus) & Verification (internal focus) view
Defect prevention		Both, mostly indirectly
	Requirement-related	Validation, indirectly
	Other defect prevention (Project Plan)	Verification indirectly
	Formal specification	Validation, indirectly
Defect Reduction	Testing type	Both, but mostly verification
	Unit	Verification
	integration	Both, more verification
	system	Both
	acceptance	Both, more validation
	beta	Validation
Defect Containment		Both, but mostly validation
	Operation	Validation
	Design & implementation	Both, but mostly verification

SOFTWARE QUALITY ENGINEERING (SQE)

- ❑ To meet or exceed these quality expectations through the selected and execution of appropriate QA activities while **minimizing the cost** and other **project risks** under the project constraints
- ❑ The SQE process forms an integral part of the overall software engineering process, where other concerns, such as **cost** and **schedule** are also considered and managed.

SQE activities-Generic Testing Process (Systematic testing based on formal models)

- ❑ Pre-QA activities: Quality Planning/Test Planning
 - Most of the key decisions about testing are made during this stage
 - ❖ **Set specific quality goals** (high-level activities to test planning)
 - Identify quality perspective and expectation: meaningful to target customers and users
 - Select direct quality measures: quantified measure of the selected quality attributes (efficiency, reliability, usefulness target in quantified values)
 - Assess quality expectations vs. Cost: cost of achieving different quality goals

SOFTWARE QUALITY ENGINEERING (SQE)

- ❖ **Form an overall QA strategy** (low-level activities to test preparation, test case)
 - Select appropriate QA activities to perform
 - Choose appropriate quality measurements and models to provide feedback, quality assessment and improvement

Test Procedure Preparation

- **Preparing test cases (micro-level)**
 - Test case is a collection of entities and related information that allows a test to be executed or a test run to be performed
 - Test case allocation
 - Sequencing of the individual test cases **from simple to complex**

TEST CASE

Project Name:			Test Designed by: Name	
Test Case ID: FR_I0			Test Designed by: Name	
Test Priority (Low, Medium, High): Medium			Test Designed date: date	
Module Name: login session			Test Executed by: Name	
Test Title: verify login with valid username and password			Test Execution date: date	
Description:Test the website login page				
Precondition: user has valid username and password				
Dependencies: if any				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to the site 2. Enter username 3. Enter password 4. Click submit	Username: urs99 Password: 321	User should login into the application	As Expected	Pass
Post Condition: User is validated with database and successfully login to account.The account session details are logged in the database				

TEST PLAN

- A **test plan** is a **document** that describes the objectives, scope, approach, resources, schedule and focus of software testing activities.
- A test plan gives detailed testing information regarding an upcoming testing effort. In other words, a test plan is a systematic approach to testing a system and typically contains a detailed understanding of what the eventual workflow will be.
- Organizations may follow some **standard test plan outlines**, or they can have their **own customized test plan outlines**.

TEST CASE

- A **test case** is a **document** that describes an input, action, or event, and its expected results, in order to determine if a feature of an application is working correctly.
- In other words, a test case is a document specifying inputs, predicted results and a set of execution conditions for a test item.
- Different organizations may use different test case formats.
- **Note:** Test Plan is a high-level document whereas Test Case is a low-level document.

SOFTWARE QUALITY ENGINEERING (SQE)

■ Preparing test suit (macro-level)

- The collection of individual test cases that will be run in a test sequence until some stopping criteria are satisfied is called a test suite.
- Involves the construction and allocation of individual test cases in some systematic way based on the specific testing techniques used.
- Another way to obtain a test suite is through reuse of test cases for earlier versions of the same product. This kind of testing is commonly referred to as regression testing.
- In general, all the test cases should form an integrated suite, regardless of their origin, how they are derived, and what models are used to derive them.

SOFTWARE QUALITY ENGINEERING (SQE)

□ In-QA activities: Test Execution

- Executing planned QA activities and handling discovered defects
- Collect failure information: what/where/when/severity/etc.
- Documentation of testing activities to check future execution results
- Organizations use template for test execution measurements

□ Post-QA activities: Quality Measurement, Assessment & Improvement

- Follow-up activities - providing feedback and identifying improvement potentials
- “Post-QA” does not mean after the finish of QA activities. In fact, many of the measurement & analysis activities are carried out parallel to QA activities after they are started. In addition, pre-QA activities may overlap with the QA activities as well.

TESTING TEAMS: ORGANIZATION & MANAGEMENT

- ❑ Customers and users, who may also serve as testers informally for usability or beta testing
- ❑ Independent professional testing organizations as trusted intermediary between software vendors and customers
- ❑ Testers and testing teams can be organized into various different structures:
 - **Vertical model:** would recognize around a product, where dedicated people perform one or more testing tasks for the product
 - **Horizontal model:** performs one kind of testing for many different products within the organization
 - **Mixed model:** often used in large software organizations that combine both horizontal and vertical model together in the testing process.

REFERENCES

- ❑ Software Testing And Quality Assurance – Theory and Practice - Kshirasagar Naik & Priyadarshi Tripathy
- ❑ Software Quality Engineering: Testing, Quality Assurance and Quantifiable Improvement - Jeff Tian